

Lab 2

Cost estimation using Linear Regression

→ Python implementation

1. importing required libraries

- pandas is used for handling & manipulating datasets in tabular format.
- numpy is used for numerical computations
- matplotlib.pyplot is used for creating graphs & visualizations
- seaborn is used for advanced data visualization with better aesthetics.

2. loading the dataset

```
df = pd.read_csv("copy of house-price-regression-dataset.csv")
```

```
df.head()
```

- pd.read_csv() loads the dataset stored in csv format.
- the dataset is stored in variable df (data frame)
- df.head() displays the first 5 rows of the dataset to understand the structure.

3. Data cleaning

```
df.isnull().sum()
```

```
df = df.drop_duplicates()
```

Explanation

- isnull().sum() checks for missing values in each column of the dataset.
- drop_duplicates() removes duplicate rows if they exist.
- Data cleaning ensures the dataset is accurate before training the model.

4. Histogram Visualization

```
plt.hist(df["House-price"], bins = 30)
plt.title("Histogram of house price")
plt.xlabel("House price")
plt.ylabel("frequency")
plt.show()
```

Explanation

- plt.hist() creates a histogram to visualize the distribution of house price.
- bins = 30 divides the data into 30 intervals.
- x label, ylabel, & title improve the readability of the graph.
- plt.show() displays the plot.

6. Boxplot Visualization

```
plt.boxplot(df["House-price"])
plt.title("Boxplot of house price")
plt.show()
```


explanation

- `plt.boxplot()` creates a boxplot.
- It helps identify outliers, median, & spread of the data.
- The boxplot is used for understanding the range of house prices.

6.6 Feature selection

from sklearn.feature_selection import SelectKBest
 from sklearn.feature_selection import f_regression

`X = df.drop("House-Price", axis=1)`

`y = df["House-Price"]`

`selector = SelectKBest(score_func=f_regression,
 k="all")`

`selector.fit(X, y).`

explanation:

- feature selection helps identify the most important variables affecting house price.
- `X` contains the input features
- `y` contains the target variable (House-Price)
- `SelectKBest` selects the best features based on statistical tests.
- `f_regression` calculates correlation between features & target variable

d. 7. Feature scaling
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

Explanation:

- feature scaling ensures all features are within the same range.
- MinMax scaler scales values between 0 & 1.
- this helps improve model performance & stability.

d. 8. Train - Test split

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size = 0.2, random_state = 42)
Ensures reproducibility of results.

d. 9. Linear Regression Model

from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train)
y_pred = model.predict(X_test)

Explanation:

- Linear regression fit () train
- predicts ()

f. 10. Actual

plt.

plt.x
plt.y
plt.

plt.

exp

•

as

•

"

i.


```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Explanation:

- LinearRegression() creates the regression model.
- fit() trains the model using training data.
- predict() generates predictions for the test data.

f. 10. Actual vs Predicted Plot

```
plt.scatter(y_test, y_pred)
```

```
plt.xlabel("Actual Price")
```

```
plt.ylabel("Predicted Price")
```

```
plt.title("Actual vs Predicted house price")
```

```
plt.show()
```

Explanation:

- plt.scatter() creates a scatter plot comparing actual & predicted values.
- If the model performs well, points should be close to the diagonal line.
- This plot helps visually evaluate model accuracy.

11. Evaluation Metrics

i. MSE (Mean Squared Error)

formula

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

no. of observations $\swarrow$ $\searrow$ $\begin{matrix} \nearrow \text{Actual value} \\ \searrow \text{Predicted value} \end{matrix}$

d. R^2 score (coefficient of determination)

$$R^2 = 1 - \frac{\sum_{i=1}^n (y - \hat{y})^2}{\sum_{i=1}^n (y - \bar{y})^2}$$

$\nearrow$ mean of predicted value
 $\downarrow$ mean of actual values

$R^2 = 1 \rightarrow$ perfect prediction

$R^2 = 0 \rightarrow$ model performs like random guessing

$R^2 < 0 \rightarrow$ model performs worse than baseline